Bingeling

MongoDB Video Streaming Case Study

24BDS0367 Upayan Mazumder

CSE302L Database Systems

22 October 2026

Contents

1. Introduction 3
2. Domain analysis 3
3. Choice of database 3
4. Data model 3
   4.1. Sample seed documents 4
5. Data population 4
6. Simple questions 5
   6.1. Q1: List all Thriller movies released after 2018, newest first. 5
   6.2. Q2: How many users have an active premium subscription? 6
   6.3. Q3: Which languages are in the catalogue? 6
   6.4. Q4: Show the 10 most recent watch events of one user. 6
   6.5. Q5: Find titles that feature a given actor. 7
7. Complex questions 7
   7.1. Q6: What are the top 5 genres by total watch minutes in each country? 7
   7.2. Q7: What is the completion rate of each movie, with average rating? 8
   7.3. Q8: Which users binge-watched (3 or more episodes of one series on one day)? 9
   7.4. Q9: What is the monthly revenue and churn per plan? 10
   7.5. Q10: How does device usage change by hour of day? 12
8. Insights 13
9. Conclusion and limitations 13
10. References 13

1. Introduction

Bingeling models a video streaming service with MongoDB 7. It stores catalogue titles, user subscriptions, rating-derived viewing proxies, reviews, plans, and generated payments. This course case study uses published catalogue and rating records where marked. It does not operate a real streaming service. The objective is to answer five simple and five complex database questions with reproducible queries and captured results [1]–[4].

2. Domain analysis

Video services need catalogue discovery, viewing history, subscriber state, and billing records. Subscribers need recent sessions. Content teams need viewing measures by title and region. Finance needs dated payments and cancellations. Product teams need completion and device-use measures [1].

Market figures have different definitions. The Government of India reported INR 9,200 crore in Indian video subscription revenue for 2024, citing FICCI-EY [7]. FICCI-EY counted 216 million paid video subscriptions across 143 million households in 2025 [6]. These are market context, not measurements of Bingeling. Netflix reported 96 billion viewing hours for July–December 2025. That figure covers Netflix, not this dataset [5].

ZEE5 reported 105 billion watch minutes in FY25 and said connected devices supplied 64% of premium-content watch time. These figures describe ZEE5, not all Indian viewers [14].

Streaming services can earn from subscriptions, ads, rentals, or combinations. This model covers subscriptions and simulated payments. Its watch events are rating-based viewing proxies or explicit synthetic scenarios, not observed video plays. It stores no video files or streaming bandwidth. The source distributions in [1] describe IMDb metadata, not an Indian platform sample.

3. Choice of database

MongoDB documents hold each title with its genres, cast, and bounded seasons. A title page can read these fields together. A user document holds a small profile list and the current subscription. MongoDB also supports aggregation pipelines for joins, time grouping, and ranking [8]. SonyLIV reported improved content-management search after its move to MongoDB Atlas. That vendor case study concerns a content system. It does not prove that MongoDB serves video bytes [10].

Reference the growing `watch_events`, `reviews`, and `payments` collections rather than embedding them in users or titles. Embedded unbounded histories would grow documents and could hit the 16 MB BSON limit [9]. The copied `watch_events.titleName` aids display but becomes stale if a title changes name. Cross-document joins and updates cost work. A relational database could enforce more relationships with foreign keys. MongoDB is a fit for this course catalogue and event analysis, not a claim of universal superiority [1], [2], [8].

4. Data model

The database is `bingeling`. Each document uses `_id`. Referenced IDs are BSON `ObjectId` values in Extended JSON seed files. Timestamps are BSON dates encoded as Extended JSON `$date`. The six collections are:

Collection	Role	Main relationships
`users`	Generated identities, country, current subscription	`_id` joins events, reviews, payments
`titles`	Movies and series, cast and catalogue metadata	`_id` joins events and reviews
`watch_events`	One generated viewing session per document	`userId`, `titleId`; copied `titleName`
`reviews`	Selected MovieLens rating records, with empty text	`userId`, `titleId`
`plans`	Plan price in INR	Plan key joins users and payments
`payments`	Generated billing period and paid amount	`userId`, plan key

Movies have `runtimeMin`. Series have `seasons[].episodes[]` with episode runtimes. Events can identify a season and episode. Plans store `priceInr`; payment records store `amountInr`, `paidAt`, `periodStart`, and `periodEnd`. A payment is a generated record, not proof of a real transfer. The model embeds bounded title details and a user's current subscription. It references growing session, review, and payment histories [2].

The loader runs `scripts/schema.js`, which installs schema validators and indexes after import. Its indexes include unique `users.email`, `titles` by genre and release year, text search over title and cast names, `watch_events` by user and start time and by title, unique `(titleId, userId)` reviews, and payment indexes by plan or user and period start. These index definitions support the access patterns; this report does not claim measured speedups.

4.1. Sample seed documents

The following examples come directly from the JSON arrays. The examples show stored types and field names, not a separate hand-written schema.

4.1.1. `users`


```
{"_id": {"$oid": "010000000000000000000001c"}, "sourceUserId": 28, "name": "Meera Singh", "email": "bingeling-000028@example.com",
"country": "IN", "timezone": "Asia/Kolkata", "joinedAt": {"$date": "1990-01-01T00:00:00Z"}, "profiles": [{"name": "Main", "kids":
false}], "subscription": {"plan": "standard", "status": "active", "startedAt": {"$date": "2025-05-01T00:00:00Z"}, "endedAt": null},
"identityOrigin": "synthetic; sourceUserId is anonymous MovieLens ID"}
```


4.1.2. `titles`


```
{"_id": {"$oid": "020000000000000000009a8"}, "type": "movie", "name": "Uncut Gems", "releaseYear": 2019, "productionCountries":
["United States"], "language": "en", "languageOrigin": "synthetic-country-proxy", "genres": ["Drama", "Thriller"], "maturity": "A",
"maturityOrigin": "mapped-netflix-rating", "cast": [{"name": "Adam Sandler"}, {"name": "LaKeith Stanfield"}, {"name": "Kevin
Garnett"}, {"name": "Julia Fox"}, {"name": "Idina Menzel"}, {"name": "Eric Bogosian"}, {"name": "Judd Hirsch"}, {"name": "Abel
Tesfaye"}], "source": {"dataset": "netflix-shows-2021", "showId": "s2472", "netflixRating": "R", "listedIn": "Dramas, Thrillers",
"movieLensMovieId": 209965, "imdbId": "tt5727208"}, "runtimeMin": 135}
```


4.1.3. `watch_events`


```
{"_id": {"$oid": "030000000000000000000001"}, "userId": {"$oid": "010000000000000000000571d"}, "titleId": {"$oid":
"020000000000000000000c0"}, "titleName": "Wind River", "device": "web", "startedAt": {"$date": "2021-03-25T00:04:23Z"},
"minutesWatched": 107, "completed": true, "origin": "movielens-rating-proxy", "sourceMovieId": 175569, "sourceRating": 3.5}
```


4.1.4. `reviews`


```
{"_id": {"$oid": "040000000000000000000001"}, "userId": {"$oid": "010000000000000000000571d"}, "titleId": {"$oid":
"020000000000000000000c0"}, "rating": 3.5, "text": "", "createdAt": {"$date": "2021-03-25T00:04:23Z"}, "origin": "movielens-rating",
"sourceMovieId": 175569}
```


4.1.5. `plans`


```
{"_id": "basic", "name": "basic", "priceInr": 199, "screens": 1, "maxQuality": "HD"}
```


4.1.6. `payments`


```
{"_id": {"$oid": "050000000000000000000001"}, "userId": {"$oid": "010000000000000000000001c"}, "plan": "standard", "amountInr": 499,
"paidAt": {"$date": "2025-05-01T00:00:00Z"}, "periodStart": {"$date": "2025-05-01T00:00:00Z"}, "periodEnd": {"$date":
"2025-06-01T00:00:00Z"}, "origin": "synthetic"}
```


5. Data population

The checked-in arrays contain the following actual record counts at report-build time. Counts come from `data/<collection>.json`; they are not market estimates.

Collection	Records
users	2,000
titles	500
watch_events	50,000
reviews	5,000
plans	3
payments	33,776

Source archive	SHA-256 from manifest
ml-32m.zip	e4a68655d7386b8f95f2f2424b2ff975dfdd15ffd59e0d864a14dca43e99d6ee
netflix-shows.zip	1e5477bbbe868845be3b852d80eec2f1da6570340860668549a6dd5da0b00054

The generator in `scripts/generate_data.py` uses fixed seed `367`. It selects 400 Netflix-listed movies matched to MovieLens links and 100 Netflix-listed series. The source archives and SHA-256 values are in `data/manifest.json`. The loader imports the Extended JSON arrays and applies validators and indexes. MovieLens anonymous rating IDs link synthetic user names to selected title ratings. Review scores and timestamps come from MovieLens, while review text is empty. Many watch-event timestamps come from those ratings and are **not** observed watch starts. The script converts rating at least 3 into `completed` and a full movie runtime. It converts a lower rating into a half-runtime session. Other events, series episodes, languages inferred from country, devices, subscriptions, and payments are generated. Actor names and catalogue labels come from Netflix metadata [11]–[13].

The Netflix titles source is a September 2021 CC0 snapshot [13]. MovieLens 32M was generated in October 2023. It includes anonymous ratings, not demographics or watch logs [12]. GroupLens permits research use and transformed redistribution only under its stated terms. It requires attribution and prohibits commercial or revenue-bearing use without permission. Those terms also apply to this derived seed data. This report does not imply GroupLens or University of Minnesota endorsement [12], [15].

Title source labels: `netflix-shows-2021`: 500. Watch-event origin labels: `movielens-rating-proxy`: 49,997, `synthetic-binge-scenario`: 3. The source archives are not an up-to-date full platform inventory. Neither a MovieLens rating nor a linked user ID proves that anyone watched a title on Bingeling.

The fixed demo user has ObjectId `0100000000000000000001c`. The fixed demo actor is `Adam Sandler`. These values are selection anchors, not real customer identities. The population screenshot below shows a captured terminal count command and its result.

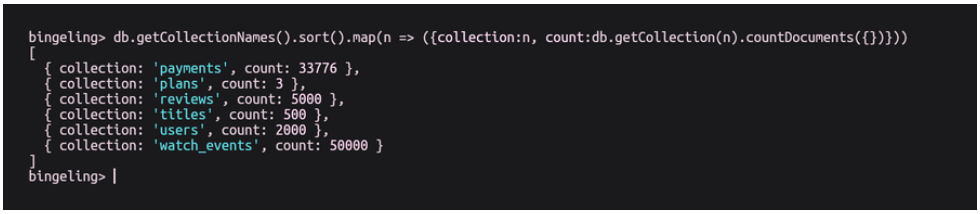


Figure 1: Population command and database counts

6. Simple questions

For every question, the JavaScript block below is copied directly from the named query file at build time. The terminal excerpt comes from the saved output. The PNG is a captured command-and-output image. The full saved output is in `docs/query-output/`. No query result is simulated in this report.

6.1. Q1: List all Thriller movies released after 2018, newest first.

Explanation. Filter the movie catalogue by genre and release year. Sort newer releases first.

Query source: `queries/simple/q01-thrillers-after-2018.js`

```
// Q1: List all Thriller movies released after 2018, newest first.
// Intent: Filter the catalogue by type, genre and release year.
// Collections: titles.
const filter = { type: 'movie', genres: 'Thriller', releaseYear: { $gt: 2018 } };
printjson({ totalMatches: db.titles.countDocuments(filter), showingNewest: 10 });
db.titles
  .find(filter, { name: 1, releaseYear: 1, genres: 1, language: 1 })
  .sort({ releaseYear: -1, name: 1 })
  .limit(10)
  .forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q01-thrillers-after-2018.txt`. The screenshot shows the command and the output.

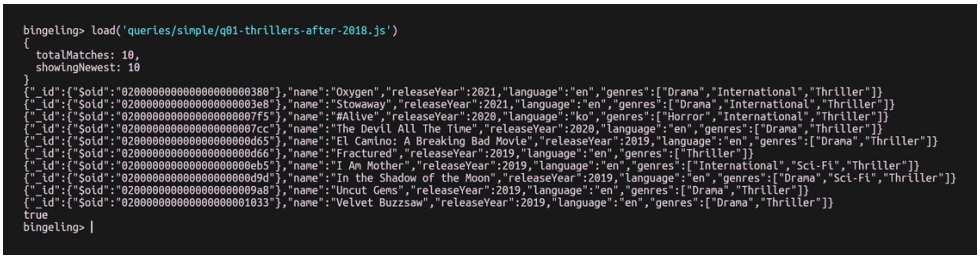


Figure 2: Q1 captured command and output

Insight and scope. The generator intentionally selects recent linked Thrillers. Their presence is not proof of demand or an unbiased catalogue sample. The screenshot above contains the observed records.

6.2. Q2: How many users have an active premium subscription?

Explanation. Count the users whose current plan is premium and whose current status is active.

Query source: `queries/simple/q02-active-premium-count.js`

```
// Q2: How many users have an active premium subscription?
// Intent: Count users whose subscription is both premium and active.
// Collections: users.
printjson({
  activePremiumUsers: db.users.countDocuments({
    'subscription.plan': 'premium',
    'subscription.status': 'active'
  })
});
```

Output file: `docs/query-output/q02-active-premium-count.txt`. The screenshot shows the command and the output.



Figure 3: Q2 captured command and output

Insight and scope. The generator assigns each user a plan and status. This count measures the current synthetic snapshot, not subscriber growth. The screenshot above contains the observed records.

6.3. Q3: Which languages are in the catalogue?

Explanation. List distinct language codes stored on title documents.

Query source: `queries/simple/q03-catalogue-languages.js`

```
// Q3: Which languages are in the catalogue?
// Intent: List unique title languages alphabetically.
// Collections: titles.
printjson(db.titles.distinct('language').sort());
```

Output file: `docs/query-output/q03-catalogue-languages.txt`. The screenshot shows the command and the output.



Figure 4: Q3 captured command and output

Insight and scope. The generator infers language from catalogue country. These codes are proxies, not confirmed audio-track languages or viewing shares. The screenshot above contains the observed records.

6.4. Q4: Show the 10 most recent watch events of one user.

Explanation. Use the fixed demo user from the manifest. Sort that user's events by start time and limit the result to ten.

Query source: `queries/simple/q04-recent-watches.js`

```
// Q4: Show the 10 most recent watch events of one user.
// Intent: Use the fixed demo user anchor and present their latest activity.
// Collections: watch_events; anchor from data/manifest.json.
const demoUserId = ObjectId(
  JSON.parse(require('fs').readFileSync('data/manifest.json', 'utf8')).demoUserId
);
printjson({ userId: demoUserId, showingNewest: 10 });
db.watch_events
  .find(
    { userId: demoUserId },
    {
      titleName: 1,
```

```
    titleId: 1,
    startedAt: 1,
    minutesWatched: 1,
    completed: 1,
    device: 1,
    season: 1,
    episode: 1,
    origin: 1
  }
)
.sort({ startedAt: -1, _id: -1 })
.limit(10)
.forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q04-recent-watches.txt`. The screenshot shows the command and the output.



Figure 5: Q4 captured command and output

Insight and scope. Many recorded start times are MovieLens rating times, not watched-at times. The result demonstrates sorting, not true recent viewing. The screenshot above contains the observed records.

6.5. Q5: Find titles that feature a given actor.

Explanation. Use the fixed demo actor from the manifest. Match the name in each title's embedded cast.

Query source: `queries/simple/q05-titles-by-actor.js`

```
// Q5: Find titles that feature a given actor.
// Intent: Match an actual credited actor against the embedded cast array.
// Collections: titles; actor anchor from data/manifest.json.
const demoActor = JSON.parse(require('fs').readFileSync('data/manifest.json', 'utf8')).demoActor;
const filter = { cast: { $elemMatch: { name: demoActor } } };
printjson({ actor: demoActor, totalMatches: db.titles.countDocuments(filter), showingFirst: 10 });
db.titles
  .find(filter, { name: 1, type: 1, releaseYear: 1, cast: 1 })
  .sort({ name: 1 })
  .limit(10)
  .forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q05-titles-by-actor.txt`. The screenshot shows the command and the output.



Figure 6: Q5 captured command and output

Insight and scope. This result checks a cast-based browse operation, not the actor's popularity. The screenshot above contains the observed records.

7. Complex questions

For every question, the JavaScript block below is copied directly from the named query file at build time. The terminal excerpt comes from the saved output. The PNG is a captured command-and-output image. The full saved output is in `docs/query-output/`. No query result is simulated in this report.

7.1. Q6: What are the top 5 genres by total watch minutes in each country?

Explanation. Join watch events to titles and users. Group watch minutes by country and genre, then rank genres within each country.

Query source: `queries/complex/q06-top-genres-by-country.js`

```
// Q6: What are the top 5 genres by total watch minutes in each country?
// Intent: Attribute each watch's minutes to every genre of its title and rank within country.
// Collections: watch_events, users, titles.
const countries = db.watch_events
  .aggregate([
    { $lookup: { from: 'users', localField: 'userId', foreignField: '_id', as: 'user' } },
    { $unwind: '$user' },
    { $lookup: { from: 'titles', localField: 'titleId', foreignField: '_id', as: 'title' } },
    { $unwind: '$title' },
    { $unwind: '$title.genres' },
```

```
    {
      $group: {
        _id: { country: '$user.country', genre: '$title.genres' },
        totalMinutes: { $sum: '$minutesWatched' },
        watchEvents: { $sum: 1 }
      }
    },
    {
      $group: {
        _id: '$_id.country',
        topGenres: {
          $topN: {
            n: 5,
            sortBy: { totalMinutes: -1, '_id.genre': 1 },
            output: {
              genre: '$_id.genre',
              totalMinutes: '$totalMinutes',
              watchEvents: '$watchEvents'
            }
          }
        }
      }
    },
    { $project: { _id: 0, country: '$_id', topGenres: 1 } },
    { $sort: { country: 1 } }
  ])
  .toArray();
printjson({ countries: countries.length, genresPerCountry: 5 });
countries.forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q06-top-genres-by-country.txt`. The screenshot shows the command and the output.

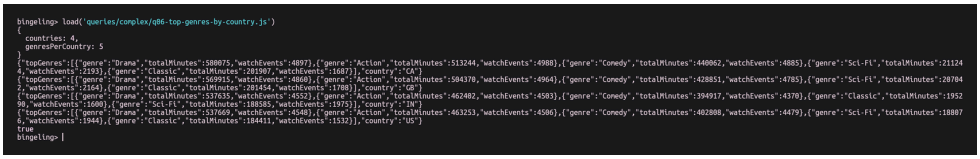


Figure 7: Q6 captured command and output

Insight and scope. Countries are assigned to anonymous rating IDs. Multi-genre titles add their minutes to each genre, so genre totals overlap. The screenshot above contains the observed records.

7.2. Q7: What is the completion rate of each movie, with average rating?

Explanation. For each movie, aggregate its watch events and independently average its reviews. An unwatched movie has no completion percentage.

Query source: `queries/complex/q07-completion-vs-rating.js`

```
// Q7: What is the completion rate of each movie, with average rating?
// Intent: Count completed movie watches and independently average title reviews (no fan-out).
// Collections: titles, watch_events, reviews.
const movies = db.titles
  .aggregate([
    { $match: { type: 'movie' } },
    {
      $lookup: {
        from: 'watch_events',
        let: { titleId: '$_id' },
        pipeline: [
          { $match: { $expr: { $eq: ['$titleId', '$$titleId'] } } },
          {
            $group: {
              _id: null,
              watches: { $sum: 1 },
              completedWatches: { $sum: { $cond: ['$completed', 1, 0] } }
            }
          }
        ],
        as: 'watchStats'
      }
    },
    {
      $lookup: {
        from: 'reviews',
        let: { titleId: '$_id' },
        pipeline: [
          { $match: { $expr: { $eq: ['$titleId', '$$titleId'] } } },
          { $group: { _id: null, averageRating: { $avg: '$rating' }, reviewCount: { $sum: 1 } } }
        ],
        as: 'reviewStats'
      }
    },
    {
      $set: {
        watchStats: { $first: '$watchStats' },
```

```
      reviewStats: { $first: '$reviewStats' }
    }
  },
  {
    $project: {
      _id: 0,
      title: '$name',
      releaseYear: 1,
      watches: { $ifNull: ['$watchStats.watches', 0] },
      completedWatches: { $ifNull: ['$watchStats.completedWatches', 0] },
      completionRatePct: {
        $cond: [
          { $gt: ['$watchStats.watches', 0] },
          {
            $round: [
              {
                $multiply: [
                  { $divide: ['$watchStats.completedWatches', '$watchStats.watches'] },
                  100
                ]
              },
              2
            ]
          },
          null
        ]
      },
      reviewCount: { $ifNull: ['$reviewStats.reviewCount', 0] },
      averageRating: { $round: ['$reviewStats.averageRating', 2] }
    }
  },
  { $sort: { completionRatePct: -1, title: 1 } }
])
  .toArray();
printjson({ totalMovies: movies.length, showingFirst: Math.min(20, movies.length) });
movies.slice(0, 20).forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q07-completion-vs-rating.txt`. The screenshot shows the command and the output.



Figure 8: Q7 captured command and output

Insight and scope. Many completed flags use the same MovieLens ratings as the reviews. The association is built into the seed and cannot show causation. The screenshot above contains the observed records.

7.3. Q8: Which users binge-watched (3 or more episodes of one series on one day)?

Explanation. Group completed series sessions by user, series, and the user's local calendar day. Count distinct episodes, then show up to 25 qualifying groups.

Query source: `queries/complex/q08-binge-watchers.js`

```
// Q8: Which users binge-watched 3 or more distinct episodes of one series on one local day?
// Intent: Deduplicate completed episodes before counting, using each user's IANA time zone.
// Collections: watch_events, users, titles.
const result = db.watch_events
  .aggregate([
    { $match: { completed: true, season: { $exists: true }, episode: { $exists: true } } },
    { $lookup: { from: 'users', localField: 'userId', foreignField: '_id', as: 'user' } },
    { $unwind: '$user' },
    { $lookup: { from: 'titles', localField: 'titleId', foreignField: '_id', as: 'title' } },
    { $unwind: '$title' },
    { $match: { 'title.type': 'series' } },
    {
      $set: {
        localDay: {
          $dateTrunc: {
            date: '$startedAt',
            unit: 'day',
            timezone: '$user.timezone'
          }
        }
      }
    },
```

```
    {
      $group: {
        _id: {
          userId: '$userId',
          titleId: '$titleId',
          localDay: '$localDay',
          season: '$season',
          episode: '$episode'
        },
        name: { $first: '$user.name' },
        timezone: { $first: '$user.timezone' },
        title: { $first: '$title.name' }
      }
    },
    {
      $group: {
        _id: { userId: '$_id.userId', titleId: '$_id.titleId', localDay: '$_id.localDay' },
        user: { $first: '$name' },
        timezone: { $first: '$timezone' },
        series: { $first: '$title' },
        distinctEpisodes: { $sum: 1 },
        episodes: { $push: { season: '$_id.season', episode: '$_id.episode' } }
      }
    },
    { $match: { distinctEpisodes: { $gte: 3 } } },
    {
      $project: {
        _id: 0,
        userId: '$_id.userId',
        titleId: '$_id.titleId',
        user: 1,
        series: 1,
        localDay: '$_id.localDay',
        timezone: 1,
        distinctEpisodes: 1,
        episodes: 1
      }
    },
    { $sort: { distinctEpisodes: -1, localDay: -1, user: 1 } },
    {
      $facet: {
        matches: [{ $count: 'count' }],
        rows: [{ $limit: 25 }]
      }
    }
  ])
  .toArray()[0];
printjson({ totalBinges: result.matches[0]?.count || 0, showingTop: result.rows.length });
result.rows.forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q08-binge-watchers.txt`. The screenshot shows the command and the output.



Figure 9: Q8 captured command and output

Insight and scope. Replays do not count as new episodes. This query shows at most 25 synthetic binge groups, not every qualifying group. The screenshot above contains the observed records.

7.4. Q9: What is the monthly revenue and churn per plan?

Explanation. Group generated payments by UTC payment month and plan. Divide current-plan cancellations that month by users active at its start.

Query source: `queries/complex/q09-revenue-and-churn.js`

```
// Q9: What is the monthly revenue and churn per plan?
// Intent: Sum actual payments by UTC payment month; churn is cancellations in that month
// divided by subscribers active at its start (null when that denominator is zero).
// Collections: payments, plans, users. Months shown have at least one payment.
const monthly = db.payments
  .aggregate([
    {
      $group: {
        _id: {
          plan: '$plan',
          month: { $dateTrunc: { date: '$paidAt', unit: 'month', timezone: 'UTC' } }
        },
        revenueInr: { $sum: '$amountInr' },
        payments: { $sum: 1 }
      }
    },
    { $lookup: { from: 'plans', localField: '_id.plan', foreignField: 'name', as: 'planDetails' } },
    { $unwind: '$planDetails' },
    { $set: { nextMonth: { $dateAdd: { startDate: '$_id.month', unit: 'month', amount: 1 } } } },
    {
```

```
  $lookup: {
    from: 'users',
    let: { plan: '$_id.plan', month: '$_id.month', nextMonth: '$nextMonth' },
    pipeline: [
      { $match: { $expr: { $eq: ['$subscription.plan', '$$plan'] } } },
      {
        $facet: {
          activeAtMonthStart: [
            {
              $match: {
                $expr: {
                  $and: [
                    { $lte: ['$subscription.startedAt', '$$month'] },
                    {
                      $or: [
                        { $eq: ['$subscription.endedAt', null] },
                        { $gt: ['$subscription.endedAt', '$$month'] }
                      ]
                    }
                  ]
                }
              }
            },
            { $count: 'count' }
          ],
          cancelledDuringMonth: [
            {
              $match: {
                $expr: {
                  $and: [
                    { $eq: ['$subscription.status', 'cancelled'] },
                    { $gte: ['$subscription.endedAt', '$$month'] },
                    { $lt: ['$subscription.endedAt', '$$nextMonth'] }
                  ]
                }
              }
            },
            { $count: 'count' }
          ]
        }
      }
    ],
    as: 'subscriberStats'
  }
},
{
  $set: {
    activeAtMonthStart: {
      $ifNull: [{ $first: { $first: '$subscriberStats.activeAtMonthStart' } }, null]
    },
    cancelledDuringMonth: {
      $ifNull: [{ $first: { $first: '$subscriberStats.cancelledDuringMonth' } }, null]
    }
  }
},
{
  $set: {
    activeAtMonthStart: { $ifNull: ['$activeAtMonthStart.count', 0] },
    cancelledDuringMonth: { $ifNull: ['$cancelledDuringMonth.count', 0] }
  }
},
{
  $project: {
    _id: 0,
    month: '$_id.month',
    plan: '$_id.plan',
    planPriceInr: '$planDetails.priceInr',
    payments: 1,
    revenueInr: 1,
    activeAtMonthStart: 1,
    cancelledDuringMonth: 1,
    churnRatePct: {
      $cond: [
        { $gt: ['$activeAtMonthStart', 0] },
        {
          $round: [
            { $multiply: [{ $divide: ['$cancelledDuringMonth', '$activeAtMonthStart'] }, 100] },
            2
          ]
        },
        null
      ]
    }
  }
},
{ $sort: { month: 1, plan: 1 } }
```

```
  ])
  .toArray();
printjson({ totalPlanMonths: monthly.length, showingFirst: Math.min(20, monthly.length) });
monthly.slice(0, 20).forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q09-revenue-and-churn.txt`. The screenshot shows the command and the output.

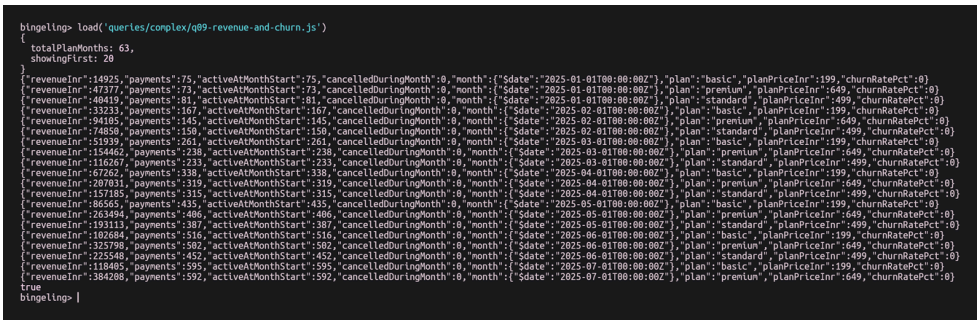


Figure 10: Q9 captured command and output

Insight and scope. The output covers months with payments. It describes generated billing, not audited revenue. A current subscription snapshot cannot track all past plan changes. The screenshot above contains the observed records.

7.5. Q10: How does device usage change by hour of day?

Explanation. Group events and watch minutes by user-local hour and device. Show six-hour dayparts and each device's cumulative hourly event share.

Query source: `queries/complex/q10-device-usage-by-hour.js`

```
// Q10: How does device usage change by hour of day?
// Intent: Compare local-hour device use, daypart totals, and each device's cumulative
// share of its own daily-profile events as hours advance (not chronological dates).
// Collections: watch_events, users (IANA timezone per user).
const usage = db.watch_events
  .aggregate([
    { $lookup: { from: 'users', localField: 'userId', foreignField: '_id', as: 'user' } },
    { $unwind: '$user' },
    { $set: { localHour: { $hour: { date: '$startedAt', timezone: '$user.timezone' } } } },
    {
      $facet: {
        hourlyByDevice: [
          {
            $group: {
              _id: { device: '$device', hour: '$localHour' },
              events: { $sum: 1 },
              minutesWatched: { $sum: '$minutesWatched' }
            }
          },
          {
            $setWindowFields: {
              partitionBy: '$_id.device',
              sortBy: { '_id.hour': 1 },
              output: {
                runningEvents: { $sum: '$events', window: { documents: ['unbounded', 'current'] } },
                deviceEvents: { $sum: '$events', window: { documents: ['unbounded', 'unbounded'] } }
              }
            }
          },
          {
            $project: {
              _id: 0,
              device: '$_id.device',
              hour: '$_id.hour',
              events: 1,
              minutesWatched: 1,
              runningEvents: 1,
              runningSharePct: {
                $round: [{ $multiply: [{ $divide: ['$runningEvents', '$deviceEvents'] }, 100] }, 2]
              }
            }
          },
          { $sort: { hour: 1, device: 1 } }
        ],
        dayparts: [
          {
            $bucket: {
              groupBy: '$localHour',
              boundaries: [0, 6, 12, 18, 24],
              output: { events: { $sum: 1 }, minutesWatched: { $sum: '$minutesWatched' } }
            }
          },
          {
            $project: {
              _id: 0,
```

```
            startHour: '$_id',
            daypart: {
              $switch: {
                branches: [
                  { case: { $eq: ['$_id', 0] }, then: 'night (00–06)' },
                  { case: { $eq: ['$_id', 6] }, then: 'morning (06–12)' },
                  { case: { $eq: ['$_id', 12] }, then: 'afternoon (12–18)' }
                ],
                default: 'evening (18–24)'
              }
            },
            events: 1,
            minutesWatched: 1
          }
        }
      ]
    }
  }
])
.toArray()[0];
printjson({ hourlyDeviceRows: usage.hourlyByDevice.length, dayparts: usage.dayparts.length });
const byHour = new Map();
for (const row of usage.hourlyByDevice) {
  if (!byHour.has(row.hour)) byHour.set(row.hour, []);
  byHour.get(row.hour).push(row);
}
for (const [hour, devices] of byHour) printjson(EJSON.stringify({ hour, devices }));
usage.dayparts.forEach((row) => printjson(EJSON.stringify(row)));
```

Output file: `docs/query-output/q10-device-usage-by-hour.txt`. The screenshot shows the command and the output.

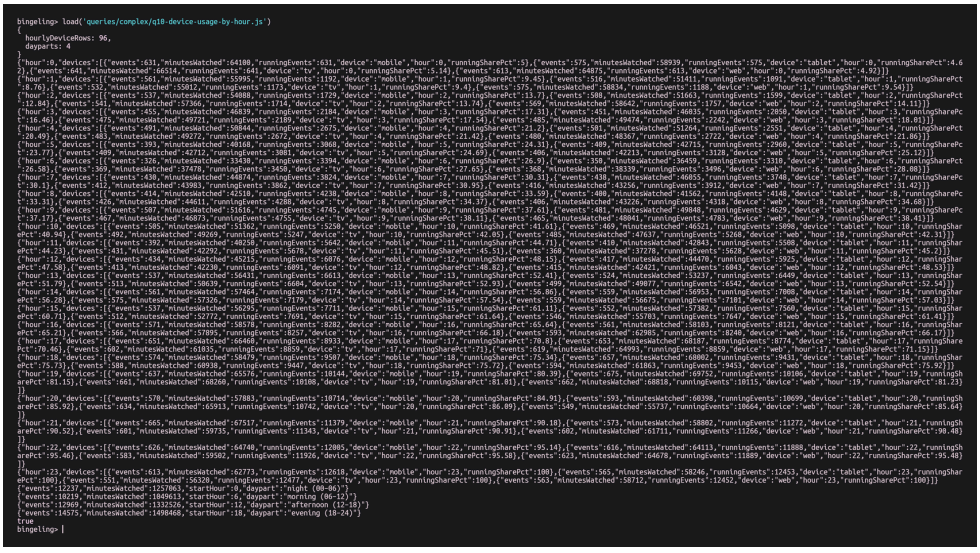


Figure 11: Q10 captured command and output

Insight and scope. Devices are assigned by the generator, and many hours come from rating timestamps. This result cannot establish real hourly demand. The screenshot above contains the observed records.

8. Insights

The catalogue queries show available titles, languages, and cast matches. Country-genre minutes, completion, binge groups, and hourly device use are properties of this constructed dataset. They are not market demand estimates. Multi-genre titles appear in several genre totals. Completion and rating are linked by the generator's rating threshold, so their association is partly built into the data. Likewise, the generator assigns countries, devices, and plan states. Monthly payments are simulated. The saved outputs above give the exact result values for each query, subject to its stated scope.

9. Conclusion and limitations

This case study demonstrates document modeling, indexed lookup designs, aggregation, and a repeatable source-to-report trail. Every query block comes from its executable source. Each screenshot shows a terminal run of its query file. Each numbered screenshot is required at build time. These checks prevent a report with missing evidence, but they cannot prove a screenshot came from a specific database state.

Netflix catalogue labels are a dated sample. MovieLens ratings are not viewing logs [12], [13]. User identities, country and time zone, devices, episodes, subscriptions, and payments are constructed. The generator derives many completion flags from ratings; this prevents independent inference from the Q7 comparison. A rating timestamp is not a watch-start time; Q4, Q8, and Q10 illustrate query mechanics rather than real viewing rhythms. A current subscription record cannot reconstruct all past plan changes. Payments do not prove collected or recognized revenue. Do not use these results for forecasts. Source terms restrict reuse of the MovieLens-derived records [1], [12], [15].

10. References

- [1] Bingeling project, "Domain analysis," docs/domain-analysis.md, 2026.
- [2] Bingeling project, "Data model," docs/data-model.md, 2026.

- [3] Bingeling project, “Fixed questions,” docs/questions.md, 2026.
- [4] Bingeling project, “Guidelines,” GUIDELINES.md, 2026.
- [5] Netflix, “What we watched: The second half of 2025.” [Online]. Available: https://about.netflix.com/news/what-we-watched-the-second-half-of-2025 Accessed: Oct. 22, 2026.
- [6] FICCI and EY, “India’s media and entertainment sector in 2025,” EY India newsroom, Mar. 2026. [Online]. Available: https://www.ey.com/en_in/newsroom/2026/03/india-s-media-and-entertainment-sector-grew-9-percent-to-inr-2-point-78-trillion-in-2025-driven-by-digital-and-live-experiences-ficci-ey-report Accessed: Oct. 22, 2026.
- [7] Government of India, Press Information Bureau, “Video subscription revenue in 2024.” [Online]. Available: https://www.pib.gov.in/PressReleasePage.aspx?PRID=2205233&lang=2&reg=3 Accessed: Oct. 22, 2026.
- [8] MongoDB, Inc., “Data modeling best practices,” MongoDB Docs. [Online]. Available: https://www.mongodb.com/docs/manual/data-modeling/best-practices/ Accessed: Oct. 22, 2026.
- [9] MongoDB, Inc., “Unbounded arrays anti-pattern,” MongoDB Docs. [Online]. Available: https://www.mongodb.com/docs/manual/data-modeling/design-antipatterns/unbounded-arrays/ Accessed: Oct. 22, 2026.
- [10] MongoDB, Inc., “SonyLIV improves CMS performance by 98 percent on Atlas.” [Online]. Available: https://www.mongodb.com/company/blog/innovation/sonyliv-improves-cms-performance-by-98-percent-on-atlas Accessed: Oct. 22, 2026.
- [11] IMDb, “IMDb non-commercial datasets.” [Online]. Available: https://data.imdb.com/non-commercial-datasets/ Accessed: Oct. 22, 2026.
- [12] GroupLens, “MovieLens 32M README and licence.” [Online]. Available: https://files.grouplens.org/datasets/movielens/ml-32m-README.html Accessed: Oct. 22, 2026.
- [13] Kaggle, “Netflix movies and TV shows,” dataset metadata. [Online]. Available: https://www.kaggle.com/api/v1/datasets/view/shivamb/netflix-shows Accessed: Oct. 22, 2026.
- [14] Deccan Chronicle, “ZEE5 clocks 105 billion watch minutes in FY25.” [Online]. Available: https://www.deccanchronicle.com/entertainment/ott/zee5-clocks-105-billion-watch-minutes-in-fy25-amid-renewed-regional-push-1898817 Accessed: Oct. 22, 2026.
- [15] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” ACM Trans. Interact. Intell. Syst., vol. 5, no. 4, Art. no. 19, 2015, doi: 10.1145/2827872.

Evidence files: `data/manifest.json`, the six `data/*.json` arrays, ten `queries/**/*.js` files, ten `docs/query-output/*.txt` files, and ten matching `screenshots/*.png`. These files fix the reproducible scope of this report.